

SarvVivah Backend Development Plan

Project Progress Summary

Metric	Status
Overall Completion	~62%
Work Done	~62%
Work Remaining	~38%
Last Updated	February 1, 2026

Architecture Overview

Tech Stack:

- Runtime: Node.js with Express.js
- Database: PostgreSQL (Supabase)
- ORM: Prisma
- Validation: Zod
- Authentication: JWT + OTP

Current Dependencies:

```
{
  "bcrypt": "^6.0.0",
  "express": "^5.2.1",
  "jsonwebtoken": "^9.0.3",
  "zod": "^4.3.6",
  "@prisma/client": "^6.19.2",
  "prisma": "^6.19.2",
  "axios": "^1.13.3",
  "cors": "^2.8.5",
  "helmet": "^8.0.0",
  "express-rate-limit": "^7.5.0",
  "winston": "^3.17.0",
  "swagger-ui-express": "^5.0.1",
  "swagger-jsdoc": "^6.2.8"
}
```

Team Assignment (3 Developers)

Developer	Focus Areas
Developer 1	Authentication, User Management, Profile Management

Developer	Focus Areas
Developer 2	Search/Matchmaking, Interests, Messaging System
Developer 3	Admin Panel, Subscriptions, Media, Reports, Analytics

PHASE 1: Foundation & Authentication (Weeks 1-2)

Developer 1 - Authentication & Core User APIs

☒ DONE - Task 1.1: Project Setup

- ☒ Express.js server setup (`index.js`)
- ☒ Prisma ORM configuration (`prisma/schema.prisma` - 310 lines)
- ☒ Database schema design (15+ tables)
- ☒ SQL schema creation (`sarvvivah.sql`)
- ☒ Environment configuration (`.env`, `.env.example`)
- ☒ Basic project structure (controllers, routes, middleware, services, utils)

☒ DONE - Task 1.2: OTP Service

- ☒ OTP generation (6-digit crypto random)
- ☒ OTP storage in database (`otp_logs` table)
- ☒ OTP verification logic
- ☒ SMS sending mock implementation
- File:** `src/services/otpService.js` (103 lines)

☒ DONE - Task 1.3: Signup Flow

- ☒ Send OTP endpoint (`POST /auth/send-otp`)
- ☒ Verify OTP endpoint (`POST /auth/verify-otp`)
- ☒ Complete signup endpoint (`POST /auth/signup`)
- ☒ Zod validation schemas
- ☒ Password hashing with bcrypt (10 rounds)
- Files:** `src/controllers/authController.js`, `src/routes/auth.js`, `src/utils/validation.js`

☒ DONE - Task 1.4: Admin Creation

- ☒ Create admin/moderator endpoint (`POST /auth/create-admin`)
- ☒ Admin secret verification
- ☒ Role-based user creation
- ☒ Conditional authentication (first admin: secret only, subsequent: ADMIN role required)
- ☒ Optional authentication middleware implementation

☒ DONE - Task 1.5: Login Flow

- ☒ Login with mobile/email + password (`POST /auth/login`)
- ☒ JWT token generation with expiration

- ☒ Login validation schema
 - ☒ Active account verification
 - **File:** `src/controllers/authController.js` (481 lines)
- ☒ DONE - Task 1.6: Extended Login Features
- ☒ Refresh token mechanism (7-day validity, token rotation)
 - ☒ Token revocation for logout (single device & all devices)
 - ☒ Session management (database-backed refresh tokens)
 - **Files:** `src/services/tokenService.js`, `src/controllers/authController.js`
- ☒ DONE - Task 1.7: Password Management
- ☒ Forgot password - send OTP (`POST /auth/forgot-password`) with rate limiting
 - ☒ Verify forgot OTP (`POST /auth/verify-forgot-otp`)
 - ☒ Reset password (`POST /auth/reset-password`) with token revocation
 - ☒ Change password (authenticated) (`POST /auth/change-password`) with token revocation
 - ☒ SMS notifications for password changes
 - **Files:** `src/controllers/authController.js`, `src/routes/auth.js`
- ☒ DONE - Task 1.8: JWT Middleware Enhancement & RBAC
- ☒ Basic JWT authentication middleware
 - ☒ Role-based authorization middleware (`authorizeRole`)
 - ☒ Permission-based authorization middleware (`authorizePermission`)
 - ☒ Resource ownership verification middleware (`checkOwnership`)
 - ☒ Token refresh logic (implemented via `tokenService` + `authController`)
 - ☒ ADMIN bypass for permission checks
 - ☒ Audit logging for authorization failures
 - ☒ Active user verification in authorization
 - **Files:** `src/middleware/auth.js`, `src/middleware/authorization.js` (275 lines)
 - **Related Files:** `src/services/tokenService.js`, `src/controllers/authController.js`

 TESTING - Developer 1 (Phase 1)

Test ID	Test Case	Type	Priority
T1.1.1	Send OTP - valid mobile number	Unit	High
T1.1.2	Send OTP - already registered mobile	Unit	High
T1.1.3	Send OTP - invalid mobile format	Unit	High
T1.2.1	Verify OTP - valid OTP	Unit	High
T1.2.2	Verify OTP - expired OTP	Unit	High
T1.2.3	Verify OTP - invalid OTP	Unit	High
T1.3.1	Signup - complete valid data	Integration	High
T1.3.2	Signup - without OTP verification	Integration	High

Test ID	Test Case	Type	Priority
T1.3.3	Signup - expired OTP verification	Integration	Medium
T1.3.4	Signup - duplicate mobile number	Integration	High
T1.4.1	Login - valid credentials (mobile)	Integration	High
T1.4.2	Login - valid credentials (email)	Integration	High
T1.4.3	Login - invalid password	Integration	High
T1.4.4	Login - non-existent user	Integration	High
T1.4.5	Login - deactivated account	Integration	Medium
T1.5.1	Create Admin - valid secret	Integration	High
T1.5.2	Create Admin - invalid secret	Integration	High
T1.6.1	JWT middleware - valid token	Unit	High
T1.6.2	JWT middleware - expired token	Unit	High
T1.6.3	JWT middleware - missing token	Unit	High

Developer 2 - Database Seeding & Enums

☒ DONE - Task 1.9: Master Data Seeding & APIs

- ☒ Create seed script for religions
- ☒ Create seed script for castes (by religion)
- ☒ Create seed script for sub-castes
- ☒ Create seed script for permissions
- ☒ Create seed script for role_permissions
- ☒ Master data API endpoints (6 endpoints)
- ☒ Authentication required for all master data routes
- Files:** `prisma/seeds/*.js`, `src/routes/masterData.js`,
`src/controllers/masterDataController.js`

☒ DONE - Task 1.10: Basic Enums

- ☒ Gender enum (Male, Female, Other)
- ☒ ProfileCreatedBy enum (Self, Parent, Guardian)
- ☒ InterestStatus enum (PENDING, ACCEPTED, REJECTED)
- ☒ Validation helpers for enums
- File:** `src/types/enums.js` (26 lines)

☒ DONE - Task 1.11: Enum Extensions

- ☒ Add MaritalStatus enum (Never Married, Divorced, Widowed, etc.)
- ☒ Add PhysicalStatus enum (Normal, Differently Abled)
- ☒ Add EmploymentType enum (Salaried, Business, Self-Employed, etc.)

- ☒ Add FamilyValues enum (Traditional, Moderate, Liberal)
- ☒ Add IncomeRange enum (ranges)
- ☒ Add PhotoVisibility enum (Public, Private, On Request)
- **Files:** Database enums in `prisma/schema.prisma`, enum data seeded via `prisma/seeds/enumMasterData.js`

 TESTING - Developer 2 (Phase 1)

Test ID	Test Case	Type	Priority
T2.1.1	Seed religions - all records created	Integration	High
T2.1.2	Seed castes - proper religion linking	Integration	High
T2.1.3	Seed sub-castes - proper caste linking	Integration	High
T2.2.1	Validate Gender enum	Unit	Medium
T2.2.2	Validate ProfileCreatedBy enum	Unit	Medium
T2.2.3	Validate InterestStatus enum	Unit	Medium
T2.3.1	Seed permissions - all created	Integration	Medium
T2.3.2	Role-permission mapping correct	Integration	Medium
T2.4.1	MaritalStatus enum validation	Unit	Medium
T2.4.2	EmploymentType enum validation	Unit	Medium

Developer 3 - Error Handling & Utilities

- ☒ DONE - Task 1.12: Error Handling Framework
 - ☒ Custom error classes (BadRequestError, UnauthorizedError, ForbiddenError, NotFoundError, ConflictError)
 - ☒ Global error handler middleware with environment-aware responses
 - ☒ Async handler wrapper utility
 - ☒ Error response standardization (success, message, statusCode)
 - ☒ Zod validation error handling
 - ☒ 404 handler for unknown routes
 - **Files:** `src/utils/errors.js`, `src/utils/asyncHandler.js`, `src/middleware/errorHandler.js`
- ☒ DONE - Task 1.13: Logging & Monitoring
 - ☒ Winston logger with file rotation and console transports
 - ☒ Request logging middleware (method, URL, status, response time, IP)
 - ☒ Error logging with stack traces
 - ☒ Structured logging utils (logAuth, logDatabase, logSecurity)
 - ☒ Separate log files (combined.log, error.log)
 - ☒ Color-coded console output with timestamps
 - **Files:** `src/config/logger.js`, `src/middleware/requestLogger.js`, `src/utils/logUtils.js`

☑ DONE - Task 1.14: Security Setup

- ☒ CORS configuration with origin whitelist
- ☒ Rate limiting (100 global, 5 auth per 15 min)
- ☒ Helmet.js for security headers (CSP, HSTS, etc.)
- ☒ Input sanitization middleware (XSS, NoSQL injection protection)
- ☒ Prisma ORM (built-in SQL injection prevention)
- ☒ Security logging for rate limit violations
- **Files:** `src/config/corsConfig.js`, `src/config/helmetConfig.js`,
`src/middleware/rateLimiter.js`, `src/middleware/sanitization.js`

☑ DONE - Task 1.15: API Documentation (Swagger/OpenAPI)

- ☒ OpenAPI 3.0.0 specification setup
- ☒ Swagger UI integration (development/staging only)
- ☒ All 12 auth endpoints documented with JSDoc
- ☒ All 6 master data endpoints documented
- ☒ JWT bearer authentication configured
- ☒ Request/response schemas with examples
- ☒ Interactive try-it-out functionality
- ☒ Component schemas (User, TokenPair, Religion, Caste, etc.)
- **Files:** `src/config/swagger.js`, `src/routes/auth.js`, `src/routes/masterData.js`
- **Access:** `http://localhost:3000/api-docs`

🔧 TESTING - Developer 3 (Phase 1)

Test ID	Test Case	Type	Priority
T3.1.1	ValidationError thrown correctly	Unit	High
T3.1.2	AuthError returns 401 status	Unit	High
T3.1.3	NotFoundError returns 404 status	Unit	High
T3.2.1	Global error handler catches all errors	Integration	High
T3.2.2	Async errors are caught properly	Integration	High
T3.3.1	Logger writes to correct file	Unit	Medium
T3.3.2	Request logging captures all fields	Integration	Medium
T3.4.1	CORS blocks unauthorized origins	Integration	High
T3.4.2	Rate limiter blocks after threshold	Integration	High
T3.4.3	Helmet headers are set correctly	Unit	Medium

📋

PHASE 2: Profile Management (Weeks 3-4) - ☒
100% COMPLETE

Developer 1 - User Profile APIs - ☒ COMPLETE

☒ DONE - Task 2.1: Personal Details CRUD

- ☒ Create personal details (`POST /users/:id/personal`)
- ☒ Update personal details (`PUT /users/:id/personal`)
- ☒ Get personal details (`GET /users/:id/personal`)
- ☒ Validation schemas for height, weight, marital status, etc.
- ☒ Additional fields: complexion, body_type, blood_group, diet_preference, drinking_habit, smoking_habit, about_me
- ☒ Authorization (self, admin, moderator)
- ☒ Audit logging for all create/update operations
- ☒ Profile completion tracking (20% weight)
- ☒ Swagger documentation
- **Files:** `src/controllers/userProfileController.js`, `src/routes/userProfile.js`, `src/utils/validation.js`
- **Documentation:** `TASK_2.1_PERSONAL_DETAILS_IMPLEMENTATION.md`, `TASK_2.1_QUICK_REFERENCE.md`

☒ DONE - Task 2.2: Caste Details CRUD

- ☒ Create caste details (`POST /users/:id/caste`)
- ☒ Update caste details (`PUT /users/:id/caste`)
- ☒ Get caste details (`GET /users/:id/caste`)
- ☒ Religion/Caste/SubCaste validation
- ☒ Authorization (Self + Admin/Moderator)
- ☒ Profile completion tracking (10% weight)
- ☒ Audit logging
- ☒ Swagger documentation
- **Files:** `src/controllers/userProfileController.js`, `src/routes/userProfile.js`
- **Documentation:** `TASK_2.2_CASTE_DETAILS_IMPLEMENTATION.md`, `TASK_2.2_QUICK_REFERENCE.md`, `TASK_2.2_SUMMARY.md`

☒ DONE - Task 2.3: Education Details CRUD

- ☒ Add education entry (`POST /users/:id/education`)
- ☒ Update education entry (`PUT /users/:id/education/:eduId`)
- ☒ Delete education entry (`DELETE /users/:id/education/:eduId`)
- ☒ Get all education entries (`GET /users/:id/education`)
- ☒ Year validation (birth_year + 15 to current_year + 5)
- ☒ Institution name validation (min 3, max 200 chars)
- ☒ Maximum 5 entries per user enforcement
- ☒ Duplicate prevention (qualification + institution + year)
- ☒ Authorization (Self + Admin only, NOT Moderator)
- ☒ Public GET access (no authentication)
- ☒ Partial updates (PATCH-style)
- ☒ Profile completion calculation (graduated: 0%, 7%, 10%)
- ☒ Audit logging for all CUD operations

- ☒ Swagger documentation with multiple examples
- **Files:** `src/controllers/userProfileController.js`, `src/routes/userProfile.js`, `src/utils/validation.js`
- **Documentation:** `TASK_2.3_EDUCATION_DETAILS_IMPLEMENTATION.md`, `TASK_2.3_QUICK_REFERENCE.md`

☒ DONE - Task 2.4: Professional Details CRUD

- ☒ Create professional details (`POST /users/:id/professional`)
- ☒ Update professional details (`PUT /users/:id/professional`)
- ☒ Partial update professional details (`PATCH /users/:id/professional`)
- ☒ Get professional details (`GET /users/:id/professional`)
- ☒ Employment type validation
- ☒ Income range validation
- ☒ Authorization (Self + Admin only)
- ☒ Profile completion tracking (10% weight)
- ☒ Audit logging
- ☒ Swagger documentation
- **Files:** `src/controllers/userProfileController.js`, `src/routes/userProfile.js`

 TESTING - Developer 1 (Phase 2)

Test ID	Test Case	Type	Priority
T2.1.1	Create personal details - valid data	Integration	High
T2.1.2	Update personal details - partial update	Integration	High
T2.1.3	Get personal details - existing user	Integration	High
T2.1.4	Create personal details - invalid height	Unit	Medium
T2.2.1	Create caste details - valid religion/caste	Integration	High
T2.2.2	Update caste details - change sub-caste	Integration	Medium
T2.2.3	Caste validation - invalid religion ID	Unit	High
T2.3.1	Add education entry - valid data	Integration	High
T2.3.2	Update education entry - existing entry	Integration	Medium
T2.3.3	Delete education entry - valid ID	Integration	High
T2.3.4	Get all education entries	Integration	Medium
T2.4.1	Create professional details - valid data	Integration	High
T2.4.2	Update professional details - income range	Integration	Medium

Developer 2 - Extended Profile APIs - ☒ COMPLETE

☒ DONE - Task 2.5: Family Details CRUD

- ☒ Create family details (POST /users/:id/family)
- ☒ Update family details (PUT /users/:id/family)
- ☒ Get family details (GET /users/:id/family)
- ☒ Siblings details validation
- ☒ Family values enum
- ☒ Authorization (Self + Admin/Moderator)
- ☒ Profile completion tracking (10% weight)
- ☒ Audit logging
- ☒ Swagger documentation
- **Files:** src/controllers/profileController.js, src/routes/profile.js

☒ DONE - Task 2.6: Horoscope Details CRUD

- ☒ Create horoscope details (POST /users/:id/horoscope)
- ☒ Update horoscope details (PUT /users/:id/horoscope)
- ☒ Get horoscope details (GET /users/:id/horoscope)
- ☒ Rasi/Nakshatra validation (27 nakshatras, 12 rasis)
- ☒ Time of birth validation
- ☒ Place of birth validation
- ☒ Authorization (Self + Admin/Moderator)
- ☒ Profile completion tracking (5% weight)
- ☒ Audit logging
- ☒ Swagger documentation
- **Files:** src/controllers/profileController.js, src/routes/profile.js
- **Documentation:** TASK_2.6_HOROSCOPE_DETAILS_SUMMARY.md, HOROSCOPE_API_QUICK_REFERENCE.md

☒ DONE - Task 2.7: Partner Preferences CRUD

- ☒ Create partner preferences (POST /users/:id/preferences)
- ☒ Update partner preferences (PUT /users/:id/preferences)
- ☒ Get partner preferences (GET /users/:id/preferences)
- ☒ Age range validation (min_age < max_age)
- ☒ Height/Weight range validation
- ☒ Religion/Caste/Education preference arrays
- ☒ Employment type preference (array support)
- ☒ Income preference range
- ☒ Location preference (JSONB field)
- ☒ Authorization (Self + Admin/Moderator)
- ☒ Profile completion tracking (5% weight)
- ☒ Audit logging
- ☒ Swagger documentation
- **Files:** src/controllers/profileController.js, src/routes/profile.js
- **Documentation:** TASK_2.7_PARTNER_PREFERENCES_SUMMARY.md, TASK_2.7_QUICK_REFERENCE.md, TASK_2.7_COMPLETION_REPORT.md

TESTING - Developer 2 (Phase 2)

Test ID	Test Case	Type	Priority
T2.5.1	Create family details - valid data	Integration	High
T2.5.2	Update family details - siblings info	Integration	Medium
T2.5.3	Get family details - existing user	Integration	Medium
T2.6.1	Create horoscope - valid rasi/nakshatra	Integration	High
T2.6.2	Update horoscope - time of birth	Integration	Medium
T2.6.3	Invalid rasi validation	Unit	Medium
T2.7.1	Create preferences - valid age range	Integration	High
T2.7.2	Update preferences - religion/caste	Integration	High
T2.7.3	Get preferences - existing user	Integration	Medium
T2.7.4	Preference matching - score calculation	Unit	High

Developer 3 - Photo Management - ☒ 100% COMPLETE

☒ DONE - Task 2.8: Photo Upload System

- ☒ Photo upload endpoint (**POST** /users/:id/photos)
- ☒ Photo deletion (**DELETE** /users/:id/photos/:photoId)
- ☒ Get user photos (**GET** /users/:id/photos)
- ☒ Set primary photo (**PUT** /users/:id/photos/:photoId/primary)
- ☒ Base64 image support
- ☒ File validation (type, size)
- ☒ Maximum 5 photos per user
- ☒ Photo visibility control (Public, Private, On Request)
- ☒ Authorization (Self + Admin/Moderator)
- ☒ Profile completion tracking (10% weight)
- ☒ Audit logging
- ☒ Swagger documentation
- Files:** src/controllers/photoController.js, src/routes/photos.js
- Documentation:** PHOTO_UPLOAD_TESTING.md
- Note:** Cloud storage integration (S3/Cloudinary) pending - currently using base64

☒ DONE - Task 2.9: Photo Visibility & Approval

- ☒ Set photo visibility (included in upload)
- ☒ Photo approval status field
- ☒ Photo approval workflow (moderator)
- ☒ Get pending approval photos (**GET** /admin/photos/pending)
- ☒ Approve photo endpoint (**PATCH** /admin/photos/:photoId/approve)
- ☒ Reject/delete photo endpoint (**DELETE** /admin/photos/:photoId)
- ☒ Audit logging for moderator actions

- ☒ Pagination support for pending photos
- ☒ FIFO ordering (oldest first)
- **Files:** `src/controllers/photoController.js`, `src/routes/admin.js`
- **Status:** Complete photo moderation system with moderator/admin access control

☒ DONE - Task 2.10: Complete Profile API + Performance Optimization

- ☒ Get complete profile (`GET /users/:id/profile`)
- ☒ Profile completion percentage calculator (enhanced)
- ☒ Profile verification status checker (`GET /users/:id/verification-status`)
- ☒ Privacy-aware data filtering (sensitive data)
- ☒ Profile readiness for matching (60% threshold)
- ☒ Profile badges system (Verified, Complete, New, Active)
- ☒ Activity status tracking
- ☒ Photo filtering (only approved with metadata)
- ☒ Education sorting (latest first)
- ☒ Nested response structure with all sections
- ☒ **NEW:** Dashboard-optimized endpoint (`GET /users/:id/completion-percentage`) - 5.96x faster
- ☒ **NEW:** Database-level caching (profile_completion_percentage field)
- ☒ **NEW:** Automatic cache invalidation on profile updates
- ☒ **NEW:** Shared utility module (`src/utils/profileCompletion.js`)
- ☒ Comprehensive Swagger documentation
- ☒ Test suite with 10 scenarios (complete profile + caching)
- ☒ Dashboard endpoint test suite (5 scenarios)
- **Files:** `src/controllers/userProfileController.js`, `src/routes/userRoutes.js`, `src/utils/profileCompletion.js`, `src/tests/completeProfileTest.js`, `src/tests/completionPercentageTest.js`
- **Documentation:** `TASK_2.10_COMPLETE_PROFILE_SUMMARY.md`, `TASK_2.10_QUICK_REFERENCE.md`, `DASHBOARD_OPTIMIZATION_SOLUTION.md`, `PROFILE_COMPLETION_CACHING.md`, `IMPLEMENTATION_SUMMARY_PROFILE_CACHING.md`

 TESTING - Developer 3 (Phase 2)

Test ID	Test Case	Type	Priority
T2.8.1	Upload photo - valid image (JPG)	Integration	High
T2.8.2	Upload photo - valid image (PNG)	Integration	High
T2.8.3	Upload photo - invalid file type	Integration	High
T2.8.4	Upload photo - file size exceeds limit	Integration	High
T2.8.5	Delete photo - valid photo ID	Integration	High
T2.8.6	Get user photos - with pagination	Integration	Medium
T2.9.1	Set visibility - PUBLIC to PRIVATE	Integration	High
T2.9.2	Approve photo - moderator action	Integration	High

Test ID	Test Case	Type	Priority
T2.9.3	Reject photo - with reason	Integration	Medium
T2.10.1	Get complete profile - all sections	Integration	High
T2.10.2	Profile completion - percentage calc	Unit	High
T2.10.3	Profile verification status	Unit	Medium
T2.10.4	Privacy filtering - sensitive data	Integration	High
T2.10.5	Profile badges - calculation	Unit	Medium
T2.10.6	Profile readiness - matching threshold	Unit	High
T2.10.7	Photo filtering - only approved	Integration	High
T2.10.8	Education sorting - latest first	Integration	Medium
T2.10.1	Get complete profile - all sections	Integration	High
T2.10.2	Profile completion - percentage calc	Unit	High
T2.10.3	Profile verification status	Unit	Medium

 PHASE 3: Search & Matchmaking (Weeks 5-6)

Developer 1 - Basic Search

- ☐ TODO - Task 3.1: Profile Listing
- ☐ Get all profiles with pagination (**GET /profiles**)
 - ☐ Filter by gender
 - ☐ Filter by age range
 - ☐ Filter by location
 - ☐ Sort options (newest, last active)
- ☐ TODO - Task 3.2: Search Filters Implementation
- ☐ Religion filter
 - ☐ Caste filter
 - ☐ Education filter
 - ☐ Profession filter
 - ☐ Income range filter
 - ☐ Marital status filter
 - ☐ Physical status filter

 TESTING - Developer 1 (Phase 3)

Test ID	Test Case	Type	Priority
---------	-----------	------	----------

Test ID	Test Case	Type	Priority
T3.1.1	Get profiles - default pagination	Integration	High
T3.1.2	Get profiles - custom page size	Integration	Medium
T3.1.3	Filter by gender - Male only	Integration	High
T3.1.4	Filter by age range - 25-30	Integration	High
T3.1.5	Sort by newest first	Integration	Medium
T3.2.1	Filter by religion	Integration	High
T3.2.2	Filter by caste (with religion)	Integration	High
T3.2.3	Combined filters - religion + age	Integration	High
T3.2.4	Filter by income range	Integration	Medium
T3.2.5	Filter by marital status	Integration	Medium

Developer 2 - Advanced Search & Matching

☐ TODO - Task 3.3: Advanced Search

- ☐ Height range filter
- ☐ Mother tongue filter
- ☐ Horoscope (Rasi/Nakshatra) filter
- ☐ Keyword search in profile
- ☐ Search by profile ID
- ☐ Search log creation (search_logs table)

☐ TODO - Task 3.4: Matchmaking Algorithm

- ☐ Partner preference matching service
- ☐ Match score calculation
- ☐ Get recommended profiles (GET /profiles/recommended)
- ☐ "New Matches" based on preferences
- ☐ Daily match suggestions

☐ TODO - Task 3.5: Profile Views & Activity

- ☐ Record profile view (create profile_views table)
- ☐ Get "Who viewed my profile" (GET /users/:id/viewers)
- ☐ Get profiles I viewed (GET /users/:id/viewed)
- ☐ Last active timestamp update

TESTING - Developer 2 (Phase 3)

Test ID	Test Case	Type	Priority
T3.3.1	Filter by height range	Integration	Medium

Test ID	Test Case	Type	Priority
T3.3.2	Filter by mother tongue	Integration	Medium
T3.3.3	Filter by rasi	Integration	Medium
T3.3.4	Keyword search - name	Integration	High
T3.3.5	Search by profile ID	Integration	High
T3.3.6	Search log created correctly	Integration	Medium
T3.4.1	Match score calculation - exact match	Unit	High
T3.4.2	Match score calculation - partial match	Unit	High
T3.4.3	Get recommended - sorted by score	Integration	High
T3.4.4	Daily match suggestions - unique	Integration	Medium
T3.5.1	Record profile view	Integration	High
T3.5.2	Get viewers - with pagination	Integration	Medium
T3.5.3	Last active timestamp updates	Integration	Medium

Developer 3 - Shortlisting

☐ TODO - Task 3.6: Shortlist Management

- ☐ Add to shortlist (POST /shortlist/:userId)
- ☐ Remove from shortlist (DELETE /shortlist/:userId)
- ☐ Get my shortlist (GET /shortlist)
- ☐ Check if shortlisted (GET /shortlist/:userId/status)
- ☐ Get "Who shortlisted me" (GET /shortlisted-by)

TESTING - Developer 3 (Phase 3)

Test ID	Test Case	Type	Priority
T3.6.1	Add to shortlist - valid user	Integration	High
T3.6.2	Add to shortlist - duplicate prevention	Integration	High
T3.6.3	Add to shortlist - self not allowed	Integration	Medium
T3.6.4	Remove from shortlist	Integration	High
T3.6.5	Get my shortlist - with details	Integration	High
T3.6.6	Get my shortlist - pagination	Integration	Medium
T3.6.7	Check shortlist status - true	Integration	Medium
T3.6.8	Check shortlist status - false	Integration	Medium
T3.6.9	Get who shortlisted me	Integration	High

PHASE 4: Communication System (Weeks 7-8)

Developer 1 - Interest System

☐ TODO - Task 4.1: Send Interest

- ☐ Send interest (POST /interests/:receiverId)
- ☐ Prevent duplicate interests
- ☐ Interest notification trigger

☐ TODO - Task 4.2: Manage Interests

- ☐ Get sent interests (GET /interests/sent)
- ☐ Get received interests (GET /interests/received)
- ☐ Accept interest (PUT /interests/:interestId/accept)
- ☐ Reject interest (PUT /interests/:interestId/reject)
- ☐ Withdraw sent interest (DELETE /interests/:interestId)

 TESTING - Developer 1 (Phase 4)

Test ID	Test Case	Type	Priority
T4.1.1	Send interest - valid receiver	Integration	High
T4.1.2	Send interest - duplicate prevention	Integration	High
T4.1.3	Send interest - self not allowed	Integration	Medium
T4.1.4	Send interest - notification triggered	Integration	Medium
T4.2.1	Get sent interests - with status	Integration	High
T4.2.2	Get received interests - pending only	Integration	High
T4.2.3	Accept interest - status update	Integration	High
T4.2.4	Reject interest - status update	Integration	High
T4.2.5	Withdraw interest - deletion	Integration	Medium
T4.2.6	Accept already rejected - error	Integration	Medium

Developer 2 - Messaging System

☐ TODO - Task 4.3: Message Service Setup

- ☐ Create message service
- ☐ Message validation (only after interest accepted)
- ☐ Send message (POST /messages/:receiverId)
- ☐ Get conversation (GET /messages/:userId)

☐ TODO - Task 4.4: Conversation Management

- ☐ Get all conversations (GET /conversations)
- ☐ Mark messages as read
- ☐ Unread message count
- ☐ Delete conversation

☐ TODO - Task 4.5: Real-time Messaging (Optional)

- ☐ Socket.io setup
- ☐ Real-time message delivery
- ☐ Online status indicator
- ☐ Typing indicator

 TESTING - Developer 2 (Phase 4)

Test ID	Test Case	Type	Priority
T4.3.1	Send message - after interest accepted	Integration	High
T4.3.2	Send message - interest not accepted	Integration	High
T4.3.3	Get conversation - chronological order	Integration	High
T4.3.4	Get conversation - pagination	Integration	Medium
T4.4.1	Get all conversations - sorted	Integration	High
T4.4.2	Mark message as read	Integration	High
T4.4.3	Unread count - accurate	Integration	High
T4.4.4	Delete conversation	Integration	Medium
T4.5.1	Socket connection established	Integration	Medium
T4.5.2	Real-time message received	Integration	Medium
T4.5.3	Online status updates	Integration	Low

Developer 3 - Notifications

☐ TODO - Task 4.6: Notification System

- ☐ Create notifications table
- ☐ Notification types (Interest, Message, Profile View, etc.)
- ☐ Get notifications (GET /notifications)
- ☐ Mark as read (PUT /notifications/:id/read)
- ☐ Notification preferences

☐ TODO - Task 4.7: Push Notifications (Optional)

- ☐ Firebase Cloud Messaging integration

- ☐ Device token management
- ☐ Push notification triggers

 TESTING - Developer 3 (Phase 4)

Test ID	Test Case	Type	Priority
T4.6.1	Create notification - interest received	Integration	High
T4.6.2	Create notification - message received	Integration	High
T4.6.3	Get notifications - sorted by date	Integration	High
T4.6.4	Mark notification as read	Integration	High
T4.6.5	Get unread notifications count	Integration	Medium
T4.6.6	Notification preferences update	Integration	Medium
T4.7.1	FCM token registration	Integration	Medium
T4.7.2	Push notification sent	Integration	Medium
T4.7.3	Push notification - device offline	Integration	Low

 PHASE 5: Admin Panel & Moderation (Weeks 9-10)

Developer 1 - User Management

☐ TODO - Task 5.1: Admin User Management

- ☐ Get all users with filters (GET /admin/users)
- ☐ View user details (GET /admin/users/:id)
- ☐ Activate/Deactivate user (PUT /admin/users/:id/status)
- ☐ Delete user (DELETE /admin/users/:id)
- ☐ Verify user profile (PUT /admin/users/:id/verify)

☐ TODO - Task 5.2: User Statistics

- ☐ Total users count
- ☐ Users by gender
- ☐ Users by religion
- ☐ New registrations (daily/weekly/monthly)
- ☐ Active users metrics

 TESTING - Developer 1 (Phase 5)

Test ID	Test Case	Type	Priority
T5.1.1	Get all users - admin access	Integration	High
T5.1.2	Get all users - non-admin blocked	Integration	High

Test ID	Test Case	Type	Priority
T5.1.3	View user details - complete data	Integration	High
T5.1.4	Deactivate user - status update	Integration	High
T5.1.5	Activate user - status update	Integration	High
T5.1.6	Delete user - soft delete	Integration	High
T5.1.7	Verify profile - flag update	Integration	Medium
T5.2.1	Total users count - accurate	Integration	Medium
T5.2.2	Users by gender - breakdown	Integration	Medium
T5.2.3	New registrations - date range	Integration	Medium

Developer 2 - Content Moderation

☐

TODO - Task 5.3: Photo Moderation

- ☐ Get pending photos (GET /admin/photos/pending)
- ☐ Approve photo (PUT /admin/photos/:id/approve)
- ☐ Reject photo (PUT /admin/photos/:id/reject)
- ☐ Bulk actions

☐

TODO - Task 5.4: Report Management

- ☐ Get all reports (GET /admin/reports)
- ☐ View report details (GET /admin/reports/:id)
- ☐ Update report status (PUT /admin/reports/:id/status)
- ☐ Take action on reported user
- ☐ Report resolution workflow

☐

TODO - Task 5.5: User Reporting

- ☐ Report user endpoint (POST /reports/:userId)
- ☐ Report reasons list
- ☐ View my reports (GET /reports/my-reports)

TESTING - Developer 2 (Phase 5)

Test ID	Test Case	Type	Priority
T5.3.1	Get pending photos - moderator	Integration	High
T5.3.2	Approve photo - status update	Integration	High
T5.3.3	Reject photo - with reason	Integration	High
T5.3.4	Bulk approve photos	Integration	Medium
T5.4.1	Get all reports - filtered	Integration	High

Test ID	Test Case	Type	Priority
T5.4.2	Update report status	Integration	High
T5.4.3	Take action - warn user	Integration	Medium
T5.4.4	Take action - deactivate user	Integration	High
T5.5.1	Report user - valid reason	Integration	High
T5.5.2	Report user - duplicate	Integration	Medium
T5.5.3	View my reports	Integration	Medium

Developer 3 - Audit & Analytics

- ☐ TODO - Task 5.6: Audit Logging
- ☐ Log all admin actions
 - ☐ Log user sensitive actions
 - ☐ Get audit logs (GET /admin/audit-logs)
 - ☐ Filter audit logs by action/user/date

- ☐ TODO - Task 5.7: Dashboard Analytics
- ☐ Registration trends
 - ☐ Subscription analytics
 - ☐ Interest/Message metrics
 - ☐ Search patterns analysis
 - ☐ Export reports (CSV/Excel)

 TESTING - Developer 3 (Phase 5)

Test ID	Test Case	Type	Priority
T5.6.1	Admin action logged	Integration	High
T5.6.2	User action logged	Integration	High
T5.6.3	Get audit logs - filtered	Integration	High
T5.6.4	Audit log - IP address captured	Unit	Medium
T5.7.1	Registration trends - daily	Integration	Medium
T5.7.2	Registration trends - monthly	Integration	Medium
T5.7.3	Interest metrics - accurate	Integration	Medium
T5.7.4	Export CSV - valid format	Integration	Medium
T5.7.5	Export Excel - valid format	Integration	Medium

 PHASE 6: Subscriptions & Payments (Weeks 11-12)

Developer 1 - Subscription Plans

- ☐ TODO - Task 6.1: Plan Management
- ☐ Create subscription_plans table
 - ☐ Get all plans (GET /plans)
 - ☐ Admin: Create plan (POST /admin/plans)
 - ☐ Admin: Update plan (PUT /admin/plans/:id)
 - ☐ Admin: Deactivate plan (DELETE /admin/plans/:id)

- ☐ TODO - Task 6.2: Feature Gating
- ☐ Define plan features (contacts/month, photo access, etc.)
 - ☐ Feature check middleware
 - ☐ Contact view limit enforcement
 - ☐ Premium features access control

 TESTING - Developer 1 (Phase 6)

Test ID	Test Case	Type	Priority
T6.1.1	Get all plans - public	Integration	High
T6.1.2	Create plan - admin	Integration	High
T6.1.3	Update plan - price change	Integration	Medium
T6.1.4	Deactivate plan	Integration	Medium
T6.2.1	Feature check - premium user	Unit	High
T6.2.2	Feature check - free user blocked	Unit	High
T6.2.3	Contact limit - reached	Integration	High
T6.2.4	Contact limit - reset monthly	Integration	Medium

Developer 2 - Payment Integration

- ☐ TODO - Task 6.3: Payment Gateway
- ☐ Razorpay/Stripe integration
 - ☐ Create payment order (POST /payments/order)
 - ☐ Payment verification webhook
 - ☐ Payment history table

- ☐ TODO - Task 6.4: Subscription Activation
- ☐ Activate subscription on payment

- ☐ Subscription renewal
- ☐ Expiry handling
- ☐ Grace period implementation

 TESTING - Developer 2 (Phase 6)

Test ID	Test Case	Type	Priority
T6.3.1	Create payment order	Integration	High
T6.3.2	Payment verification - success	Integration	High
T6.3.3	Payment verification - failure	Integration	High
T6.3.4	Payment webhook - signature valid	Unit	High
T6.3.5	Payment history recorded	Integration	Medium
T6.4.1	Subscription activated	Integration	High
T6.4.2	Subscription renewal	Integration	Medium
T6.4.3	Subscription expired - access blocked	Integration	High
T6.4.4	Grace period - access allowed	Integration	Medium

Developer 3 - Subscription Management

☐ TODO - Task 6.5: User Subscription APIs

- ☐ Get my subscription (GET /subscription)
- ☐ Subscription history (GET /subscription/history)
- ☐ Check feature access (GET /subscription/features)
- ☐ Cancel subscription

☐ TODO - Task 6.6: Admin Subscription Management

- ☐ View all subscriptions (GET /admin/subscriptions)
- ☐ Manual subscription activation
- ☐ Subscription refunds
- ☐ Revenue reports

 TESTING - Developer 3 (Phase 6)

Test ID	Test Case	Type	Priority
T6.5.1	Get my subscription - active	Integration	High
T6.5.2	Get my subscription - none	Integration	Medium
T6.5.3	Subscription history	Integration	Medium
T6.5.4	Check feature access	Integration	High

Test ID	Test Case	Type	Priority
T6.5.5	Cancel subscription	Integration	High
T6.6.1	Admin - view all subscriptions	Integration	High
T6.6.2	Admin - manual activation	Integration	Medium
T6.6.3	Admin - process refund	Integration	Medium
T6.6.4	Revenue report - date range	Integration	Medium

 PHASE 7: Advanced Features (Weeks 13-14)

Developer 1 - Email Services

- ☐ TODO - Task 7.1: Email Integration
- ☐ Email service setup (SendGrid/AWS SES)
 - ☐ Email templates (Welcome, OTP, Interest, etc.)
 - ☐ Email verification flow
 - ☐ Resend verification email

- ☐ TODO - Task 7.2: Email Notifications
- ☐ New match notification email
 - ☐ Interest received email
 - ☐ Message received email
 - ☐ Subscription expiry reminder

 TESTING - Developer 1 (Phase 7)

Test ID	Test Case	Type	Priority
T7.1.1	Welcome email sent	Integration	High
T7.1.2	OTP email sent	Integration	High
T7.1.3	Email verification - link valid	Integration	High
T7.1.4	Resend verification email	Integration	Medium
T7.2.1	New match email triggered	Integration	Medium
T7.2.2	Interest received email	Integration	Medium
T7.2.3	Subscription expiry reminder	Integration	Medium

Developer 2 - SMS Integration

- ☐ TODO - Task 7.3: SMS Gateway Integration

- ☐ SMS service integration (Twilio/MSG91)
- ☐ SMS templates
- ☐ OTP SMS (replace mock)
- ☐ Promotional SMS (opt-in)

☐ TODO - Task 7.4: Contact Sharing

- ☐ Contact view request
- ☐ Phone number masking
- ☐ Contact reveal on acceptance
- ☐ Contact view tracking

 TESTING - Developer 2 (Phase 7)

Test ID	Test Case	Type	Priority
T7.3.1	OTP SMS sent via gateway	Integration	High
T7.3.2	SMS template formatting	Unit	Medium
T7.3.3	Promotional SMS - opt-in check	Integration	Medium
T7.4.1	Contact view request	Integration	High
T7.4.2	Phone number masked	Unit	High
T7.4.3	Contact revealed on acceptance	Integration	High
T7.4.4	Contact view tracking	Integration	Medium

Developer 3 - Master Data APIs

☒ DONE - Task 7.5: Location APIs

- ☒ Get all Indian states ([GET /api/master/states](#))
- ☒ Get cities by state with search ([GET /api/master/cities?state=Karnataka&search=Bang](#))
- ☒ State validation
- ☒ City validation in state
- ☒ API integration with CountriesNow API
- ☒ In-memory caching with weekly refresh (7 days)
- ☒ Fallback to static data if API fails
- ☒ Automatic cache refresh on expiry
- **Files:** [src/services/locationService.js](#) (286 lines),
[src/controllers/masterDataController.js](#)
- **API Endpoints:** 2 public endpoints with comprehensive validation
- **Note:** Countries list and pincode lookup not needed for India-only app

☒ DONE - Task 7.6: Master Data APIs

- ☒ Get all enums ([GET /api/master/enums](#)) - 17 enum types
- ☒ Get all religions ([GET /api/master/religions](#))

- ☒ Get castes by religion (GET /api/master/castes/:religionId)
- ☒ Get sub-castes by caste (GET /api/master/sub-castes/:casteId)
- ☒ Get all master data combined (GET /api/master/all)
- ☒ Get religion hierarchy (GET /api/master/religion-hierarchy/:religionId)
- ☒ Get states (GET /api/master/states)
- ☒ Get cities by state (GET /api/master/cities?state=Karnataka)
- ☒ Active/inactive filtering
- ☒ Comprehensive validation
- ☒ Error handling for invalid IDs
- **Files:** src/controllers/masterDataController.js (388 lines), src/routes/masterData.js
- **Total Endpoints:** 8 master data endpoints
- **Enums Included:** gender, profileCreatedBy, maritalStatus, physicalStatus, employmentType, familyValues, incomeRange, photoVisibility, educationLevel, dietPreference, drinkingHabit, smokingHabit, heightRanges, ageRanges, motherTongue, rasi, nakshatra

 TESTING - Developer 3 (Phase 7)

Test ID	Test Case	Type	Priority	Status
T7.5.1	Get all states (India)	Integration	High	☒ DONE
T7.5.2	Get cities by state	Integration	High	☒ DONE
T7.5.3	Cities search with query	Integration	Medium	☒ DONE
T7.5.4	State validation	Integration	High	☒ DONE
T7.5.5	City validation in state	Integration	High	☒ DONE
T7.5.6	Location cache mechanism	Unit	Medium	☒ DONE
T7.6.1	Get all enums	Integration	High	☒ DONE
T7.6.2	Get all religions	Integration	High	☒ DONE
T7.6.3	Get castes by religion	Integration	High	☒ DONE
T7.6.4	Get sub-castes by caste	Integration	High	☒ DONE
T7.6.5	Get all master data	Integration	Medium	☒ DONE
T7.6.6	Get religion hierarchy	Integration	Medium	☒ DONE
T7.6.7	Invalid religion ID handling	Integration	Medium	☒ DONE

 PHASE 8: Testing & Deployment (Weeks 15-16)

Developer 1 - Unit Testing

☐ TODO - Task 8.1: Test Setup

- ☐ Configure Jest/Vitest

- ☐ Mock Prisma client
 - ☐ Test database setup
 - ☐ CI/CD test pipeline
- ☐ TODO - Task 8.2: Auth Tests
- ☐ OTP service tests
 - ☐ Signup flow tests
 - ☐ Login flow tests
 - ☐ JWT middleware tests
 - Current File:** `src/tests/auth.test.js` (basic structure exists - 32 lines)

 TESTING - Developer 1 (Phase 8)

Test ID	Test Case	Type	Priority
T8.1.1	Jest configuration works	Setup	High
T8.1.2	Prisma mock functional	Setup	High
T8.1.3	Test database connection	Setup	High
T8.2.1	All auth unit tests pass	Unit	High
T8.2.2	Code coverage > 80%	Metric	High

Developer 2 - Integration Testing

- ☐ TODO - Task 8.3: API Integration Tests
- ☐ Profile APIs tests
 - ☐ Search APIs tests
 - ☐ Interest/Message tests
 - ☐ Subscription tests

- ☐ TODO - Task 8.4: E2E Testing
- ☐ Complete user journey tests
 - ☐ Payment flow tests
 - ☐ Admin workflow tests

 TESTING - Developer 2 (Phase 8)

Test ID	Test Case	Type	Priority
T8.3.1	All profile API tests pass	Integration	High
T8.3.2	All search API tests pass	Integration	High
T8.3.3	All interest tests pass	Integration	High
T8.4.1	User registration to match journey	E2E	High

Test ID	Test Case	Type	Priority
T8.4.2	Payment to subscription journey	E2E	High
T8.4.3	Admin moderation workflow	E2E	Medium

Developer 3 - Deployment

☐

TODO - Task 8.5: Production Setup

- ☐ Environment configuration
- ☐ Database migration scripts
- ☐ Docker containerization
- ☐ Nginx/PM2 configuration

☐

TODO - Task 8.6: CI/CD Pipeline

- ☐ GitHub Actions workflow
- ☐ Automated testing
- ☐ Automated deployment
- ☐ Health check endpoints

☐

TODO - Task 8.7: Documentation

- ☐ API documentation (Swagger/OpenAPI)
- ☐ Postman collection
- ☐ README updates
- ☐ Database ERD diagram

TESTING - Developer 3 (Phase 8)

Test ID	Test Case	Type	Priority
T8.5.1	Docker build succeeds	Deployment	High
T8.5.2	Docker container runs	Deployment	High
T8.5.3	Database migration runs	Deployment	High
T8.6.1	GitHub Actions workflow runs	CI/CD	High
T8.6.2	Auto deploy on merge	CI/CD	High
T8.6.3	Health check endpoint returns 200	Integration	High
T8.7.1	Swagger docs accessible	Documentation	Medium
T8.7.2	Postman collection imports	Documentation	Medium

Detailed Progress Breakdown

Completed Tasks (What's Done)

Category	Task	File	Status
Setup	Express server setup	<code>index.js</code>	☒ Done
Setup	Prisma ORM & Schema	<code>prisma/schema.prisma</code>	☒ Done
Setup	Database SQL schema	<code>sarvvivah.sql</code>	☒ Done
Setup	Project folder structure	Multiple	☒ Done
Auth	Send OTP endpoint	<code>authController.js</code>	☒ Done
Auth	Verify OTP endpoint	<code>authController.js</code>	☒ Done
Auth	Signup endpoint	<code>authController.js</code>	☒ Done
Auth	Login endpoint	<code>authController.js</code>	☒ Done
Auth	Create admin endpoint (conditional auth)	<code>authController.js</code>	☒ Done
Auth	Forgot password flow (3 endpoints)	<code>authController.js</code>	☒ Done
Auth	Change password (authenticated)	<code>authController.js</code>	☒ Done
Auth	Refresh token system	<code>tokenService.js</code>	☒ Done
Auth	Logout (single & all devices)	<code>authController.js</code>	☒ Done
Auth	Zod validation schemas	<code>validation.js</code>	☒ Done
Authorization	Role-based authorization	<code>authorization.js</code>	☒ Done
Authorization	Permission-based authorization	<code>authorization.js</code>	☒ Done
Authorization	Resource ownership checks	<code>authorization.js</code>	☒ Done
Authorization	Audit logging for auth failures	<code>authorization.js</code>	☒ Done
Services	OTP generation & verification	<code>otpService.js</code>	☒ Done
Services	Token service (access & refresh)	<code>tokenService.js</code>	☒ Done
Services	Location service (states & cities)	<code>locationService.js</code>	☒ Done
Master Data	All 8 API endpoints	<code>masterDataController.js</code>	☒ Done
Master Data	Authentication on all routes	<code>masterData.js</code>	☒ Done
Master Data	Database seeding scripts	<code>prisma/seeds/*.js</code>	☒ Done
Config	Prisma client configuration	<code>prisma.js</code>	☒ Done
Config	Winston logger setup	<code>logger.js</code>	☒ Done
Config	Swagger/OpenAPI docs	<code>swagger.js</code>	☒ Done
Config	CORS configuration	<code>corsConfig.js</code>	☒ Done
Config	Helmet security headers	<code>helmetConfig.js</code>	☒ Done

Category	Task	File	Status
Utils	All enums (Gender, Marital, etc.)	enums.js	☑ Done
Utils	Custom error classes	errors.js	☑ Done
Utils	Async handler wrapper	asyncHandler.js	☑ Done
Middleware	JWT authentication	auth.js	☑ Done
Middleware	Error handler (global)	errorHandler.js	☑ Done
Middleware	Rate limiting	rateLimiter.js	☑ Done
Middleware	Input sanitization	sanitization.js	☑ Done
Middleware	Request logging	requestLogger.js	☑ Done
Photo Mgmt	Photo upload & delete	photoController.js	☑ Done
Photo Mgmt	Photo approval workflow (moderator)	photoController.js	☑ Done
Photo Mgmt	Get pending photos	photoController.js	☑ Done

File Statistics

File	Lines of Code	Status
prisma/schema.prisma	310	Complete
src/controllers/authController.js	504	Active
src/middleware/authorization.js	275	Complete
src/services/otpService.js	103	Complete
src/services/tokenService.js	289	Complete
src/routes/masterData.js	172	Complete
src/controllers/masterDataController.js	~200	Complete
src/utils/validation.js	75	Active
src/routes/auth.js	589	Complete
src/types/enums.js	26	Active
src/middleware/auth.js	50	Complete
src/config/prisma.js	12	Complete
src/config/logger.js	89	Complete
src/config/swagger.js	~300	Complete
src/middleware/errorHandler.js	150	Complete
src/utils/errors.js	45	Complete

File	Lines of Code	Status
src/controllers/photoController.js	557	Complete
src/services/locationService.js	286	Complete
src/utils/profileCompletion.js	270	Complete
sarvvivah.sql	~220	Complete

 Completion Metrics

By Feature Area

Feature Area	Completed	Total	Percentage
Project Setup	6	6	100%
Authentication & Authorization	8	8	100%
Security & Middleware	4	4	100%
Master Data & Enums	3	3	100%
API Documentation	1	1	100%
Profile Management	10	10	100%
Search & Matchmaking	0	6	0%
Interests & Messaging	0	7	0%
Admin & Moderation	1	7	14%
Subscriptions	0	6	0%
Advanced Features (Location/Master Data)	2	4	50%
Testing	0	4	0%
Deployment	0	3	0%
TOTAL	35	69	~51%

By Developer (Estimated Effort)

Developer	Tasks Done	Tasks Remaining	Completion
Developer 1 (Auth & Profiles)	14	~8	~64%
Developer 2 (Search & Master Data)	9	~11	~45%
Developer 3 (Photos & Admin)	8	~14	~36%

 Testing Summary by Phase

Phase	Dev 1 Tests	Dev 2 Tests	Dev 3 Tests	Total
Phase 1	20	10	10	40
Phase 2	13	10	12	35
Phase 3	10	13	9	32
Phase 4	10	11	9	30
Phase 5	10	11	9	30
Phase 6	8	9	9	26
Phase 7	7	8	9	24
Phase 8	5	6	8	19
Total	83	78	75	236

 Recent Major Achievements (January 2026)

Phase 2 Profile Management - 100% COMPLETE

Performance Optimization (January 29 - February 1, 2026)

- **Dashboard Optimization:** Implemented database-level caching for profile completion percentage
 - Created `profile_completion_percentage` field with automatic cache invalidation
 - Built ultra-fast dashboard endpoint: **5.96x faster** (166ms vs 989ms)
 - Updated **19+ CRUD methods** across 3 controllers with cache invalidation
 - Created shared utility module: `profileCompletion.js` (270 lines)
 - **Performance:** 139ms average for cloud database (target: <100ms for local)
 - **Files:** `src/utils/profileCompletion.js`, `src/controllers/userProfileController.js`
 - **Testing:** 15 tests passing (10 complete profile + 5 dashboard endpoint)
 - **Documentation:** 4 comprehensive guides created

Photo Management System (Complete)

- **Upload & Management:**
 - Upload photos with visibility control (Public/Private)
 - Maximum 5 photos per user with primary photo designation
 - UploadThing integration for cloud storage
 - Profile completion tracking (10% weight)
- **Moderator Workflow:**
 - Get pending photos with pagination (`GET /admin/photos/pending`)

- Approve photos with audit logging (`PATCH /admin/photos/:photoId/approve`)
- Reject/delete photos with reason tracking (`DELETE /admin/photos/:photoId`)
- FIFO queue (oldest first) for fair moderation
- **File:** `photoController.js` (557 lines)

Location Service (Complete)

- **Indian Location Data:**
 - All 36 states and union territories
 - Cities by state with search functionality
 - External API integration (CountriesNow API)
 - In-memory caching with 7-day refresh cycle
 - Fallback to static data on API failure
 - Automatic cache refresh mechanism
 - **File:** `locationService.js` (286 lines)

Master Data APIs (8 Endpoints Complete)

- Get all enums (17 types)
- Get religions with active/inactive filtering
- Get castes by religion with validation
- Get sub-castes by caste
- Get all master data combined
- Get religion hierarchy
- Get states (India)
- Get cities by state with search
- **File:** `masterDataController.js` (388 lines)

What Changed This Week

Code Changes

1. New Files Created:

- `src/utils/profileCompletion.js` - Profile completion caching utility
- `src/services/locationService.js` - Location data service
- `src/tests/completeProfileTest.js` - Complete profile test suite
- `src/tests/completionPercentageTest.js` - Dashboard endpoint tests
- `documentation/DASHBOARD_OPTIMIZATION_SOLUTION.md`
- `documentation/PROFILE_COMPLETION_CACHING.md`
- `documentation/IMPLEMENTATION_SUMMARY_PROFILE_CACHING.md`
- `documentation/PROFILE_COMPLETION_CACHING_QUICK_REFERENCE.md`

2. Database Changes:

- Added `profile_completion_percentage` field to users table
- Created index on completion percentage for fast queries
- Implemented automatic cache invalidation triggers

3. Controllers Updated:

- `userProfileController.js` - 11 methods + new dashboard endpoint
- `profileController.js` - 6 methods with cache updates
- `photoController.js` - 3 moderation methods added
- `masterDataController.js` - 2 location endpoints added

Test Coverage

- **Complete Profile Tests:** 10/10 passing ☒
 - Create/update all profile sections
 - Privacy filtering verification
 - Profile completion calculation
 - Verification status checks
- **Dashboard Endpoint Tests:** 5/5 passing ☒
 - Basic functionality
 - Performance benchmarking (5.28x faster than full profile)
 - Authorization checks
 - Cache consistency validation

Documentation Updates

- Updated Phase 2 from 95% to 100% complete
- Marked Tasks 2.2 through 2.10 as DONE
- Added Task 2.9 (Photo Approval) as complete
- Added Task 7.5 (Location APIs) as complete
- Added Task 7.6 (Master Data APIs) as complete
- Updated overall completion from 48% to 62%

Immediate Next Steps

Current Focus: Phase 3 - Search & Matchmaking

Priority Tasks for Next Week

1. Developer 1 - Basic Search:

- Implement profile listing with pagination
- Add basic filters (gender, age, location)
- Create search result endpoints

2. Developer 2 - Advanced Search:

- Build matchmaking algorithm
- Implement preference matching service
- Create recommended profiles endpoint

3. Developer 3 - Shortlisting:

- Create shortlist management endpoints
- Implement "who shortlisted me" feature
- Add shortlist status checking

Dependencies to Install

```
# Required packages (add to package.json)
npm install cors                # CORS handling
npm install helmet              # Security headers
npm install express-rate-limit  # Rate limiting
npm install winston             # Logging
npm install multer              # File uploads
npm install @aws-sdk/client-s3  # Cloud storage (optional)
npm install razorpay            # Payment gateway (optional)
npm install socket.io           # Real-time messaging (optional)
npm install nodemailer          # Email service (optional)
npm install swagger-ui-express  # API documentation
npm install jest                # Testing framework
npm install supertest           # HTTP testing
```

Last Updated: January 26, 2026 Generated from codebase analysis